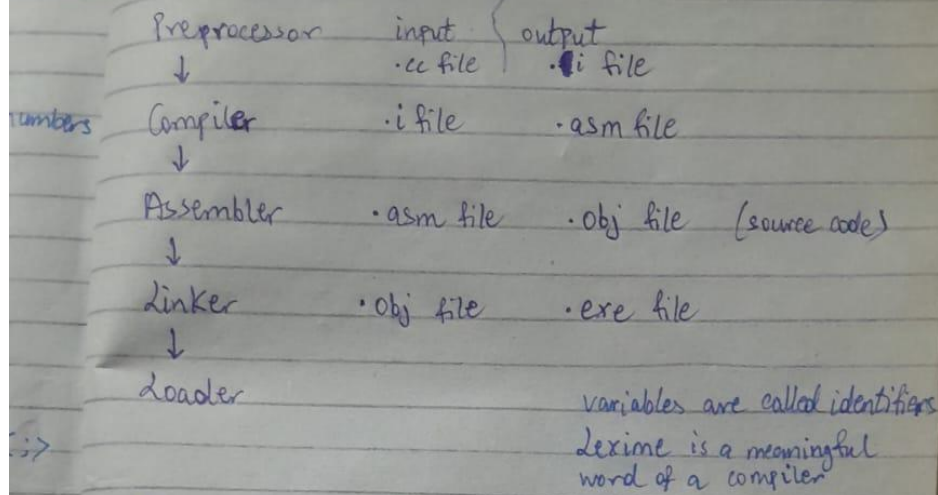


Compiler Construction



All imports for e.g. #include iostream are resolved in preprocessor. Modified source code is .I file

Interpreter executes code line by line but compiler first converts code into executable.

Phases Of Compiler

① Lexical Analysis / Scanner
outputs stream of tokens

Token: < data type, pointer to ST >

ST: Symbol table that saves identifiers

Transition diagrams

RE, RD (regular definitions)

only DFA to be implemented that are called transition diagrams.

code:

position	Lexeme	
1	position	1. float partition;
2	initial	2. float initial;
3	rate	3. float rate;
		4. partition = initial + rate * 60;

Tokens:

method 1: $\langle ID, 1 \rangle$ class 4 position in symbol table

method 2: $\langle = \rangle$ Lexeme only in case of pointers, punctuation, numbers
 $\langle op, = \rangle$

Keyword table can be made separately so that

$\langle class, pointer \rangle$ can be used for keywords.

Literal table to store string or number literals.

Total tokens in line 4 of code:

$\langle ID, 1 \rangle \langle = \rangle \langle ID, 2 \rangle \langle + \rangle \langle ID, 3 \rangle \langle * \rangle \langle 60 \rangle \langle ; \rangle$

Lexical errors such as $[int\ ab\ \$def;]$, in that
'\$' is stored in errors.txt

{All possible characters} - {set of characters in C language}
= Lexical errors

② Syntax Analysis / Parser

input: stream of tokens

output: Abstract parse tree

we CFC in form of PDA

typical syntax errors such as semicolon missing or
ill structured language

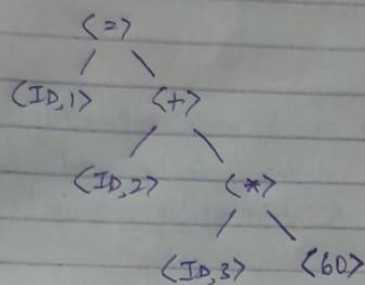
int name; $\rightarrow$ keyword
 $\downarrow \downarrow \downarrow$
keyword ID operator ID

inta;
{
 inta;
}
not an error
scope changed

Syntax Analysis adds ^{data} type column
to symbol table

scope is defined via a new symbol table.
Global variables copied in every symbol table.

parse tree:

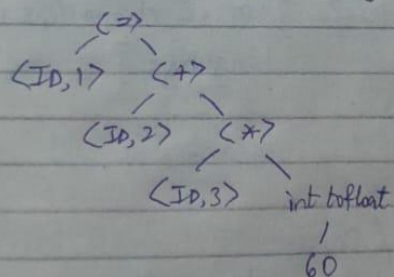


③

Semantic Analysis

output: Annotated Parse tree

Attributed grammar (FL + semantics)
type casting etc. change meaning of operators.



Semantic errors:

Array out of bounds
invalid memory access
redefinition errors

Size of pointer/variable column can be added to
symbol table in this phase or other attributes.

④ Intermediate Code Generation

output: Three Address code

Syntax directed translation

: somewhat like a
CEA

e.g.

t1 = int to float (60)

$t_2 = \langle ID, 3 \rangle * t_1$
 $t_2 = \langle ID, 2 \rangle + t_2$
 $\langle ID, 1 \rangle = t_2$

placeholders such as t_1, t_2 also
get stored in symbol table

tree read bottom to top & converted

First pass of Compiler / Front end Completed
after 4 phases. $\cdot cpp \rightarrow \cdot tac$

Code Optimization

input: $\cdot tac$ output: $\cdot tac$

e.g.

$t_1 = \langle ID, 3 \rangle * \#60.0$
 $\langle ID, 1 \rangle = \langle ID, 2 \rangle + t_1$

reduce no. of
instructions

Three Address code has some rules.

Code Generation

LDR R2, Id3
 MULF R2, R1, #60.0
 LDR R1, Id2
 ADDF R1, R1, R2
 STF Id1, R1

Machine Dependant Code Optimization

Regular Definition

operators: . \ / {} [] () * + ?
^ | " »

? : 0 or 1 occurrence

{ } : 0 - n range defined e.g. {0-3}